



Gosford High School
Preliminary Software Engineering
Assessment Task 2025

Number and Title of Task	Assessment Task 1 – Data Science Project
Topic	Fundamentals of programming
Weighting/Marks	30% (100 marks)
Due Date	Thu 10 th April 2025 (Term 1, Week 11) – During Year 11 Assessment Block
Outcomes to be Assessed (Syllabus outcomes being assessed by task) SE-11-01: Describes methods used to plan, develop, and engineer software solutions. SE-11-02: Explains how structural elements are used to develop programming code. SE-11-06: Applies tools and resources to design, develop, manage, and evaluate software. SE-11-07: Implements safe and secure programming solutions. SE-11-08: Applies language structures to refine code.	
Task Description (What you are to do) There are two components to this task. 1. Develop a data science application using Python, external Python modules open-source APIs. Include code comments, a README file and a requirements.txt file. 2. Document your project’s development process in a Markdown (.md) document.	
Additional Information (specific timing, additional resources, criteria, etc) • See detailed information for Part 1 (page 3) and Part 2 (page 4). • Marking Criteria on pages 5-10.	
Feedback (How you will receive feedback and reflect on the task) Teacher feedback will include: • Verbal formative feedback on draft responses • Individual written summative feedback • Verbal summative feedback as a class group Students will reflect on their task: • Annotating their response and conducting a reflection and evaluation (see task) • Completing a self-reflection survey (at time of submission)	
Student Acknowledgement of Academic Integrity By submitting the task for marking, I acknowledge the following: 1. The work submitted is my own work and appropriately acknowledges of all sources has been made. 2. I have not used generative AI in the construction of the task. 3. If there is an allegation of malpractice, I will be required to show my drafts/ planning to prove the task is all my own work.	

4. I am aware that the work may be submitted to plagiarism detection processes.
5. Where the work of others is used and not acknowledged, a finding of plagiarism will be made and a mark of zero awarded and I will have to resubmit the task.

Develop a Data Science Application

Part 1: Data Science Application (Practical)

Functionality:

The app should allow users to look for specific data using an API. It should detect user errors such as spelling or unexpected inputs. It should account for the use of upper- and lower-case characters. It should handle errors and exceptions gracefully and prevent run-time errors.

You are also encouraged to use Pandas to perform any cleaning of data that may be required.

- Retrieve data in a specific area using a free API (e.g. OpenWeatherMap).
- Make data available to the application.
- Display information to the user either using text or visualisations (e.g. Matplotlib).
- Record the user interaction – this can be stored in the program or a Pandas dataframe just while it runs, or permanently in a .txt file or .csv file.
- Display past interactions (while in the runtime session).

User Interface:

- The app can either have a simple command-line interface or a Graphical User Interface (GUI).
- Prompt the user to look up data using appropriate menu-options.
- Display information based on what the user requested.
- Allow the program to exit without errors.
- Allow the user to ask for 'help'.
- Handle user errors and exceptions gracefully.

API Integration:

- Use the requests library to fetch weather data from an open-source API (e.g. <https://openweathermap.org/>)
- Handle API responses (e.g., error handling and messaging)

Documentation:

- Include **comments** in your code to explain each **function** and major steps.
- Write a brief **README.md** file describing how to run the app and any dependencies.
- Include a **requirements.txt** file to make it easier for users to install.
- Each time you edit any code or documentation, provide an explanation of what you have done with a GitHub commit. You will be provided with a scaffold for this reflection.

Part 2: Data Science Application (Development Process)

Students must document their software development steps in Markdown. This should be stored as 'Documentation.md' in their GitHub repository.

Requirements Definition

Provide features, properties or behaviours the system must have to achieve its purpose. Each requirement must be verifiable.

Determining Specifications

Determine functional specifications of the system, such as data loading or data cleaning. Include what the specification is, what input is required and what output will be produced. Include use cases for each functional requirement.

Determine non-functional specifications from both the developer and user perspective.

Design

Provide clear visual representation of your program using a data dictionary, structure chart and a flowchart and algorithm for each routine and subroutine in the structure chart.

Additionally, include a Gantt chart to schedule tasks.

Development

Develop your user interface for your program and get it to function. At this point just include numbered screens and/or print statements with different text as proof that it works.

Screenshot and provide an evaluation of its functionality so far.

Integration

Integrate the open-source API into your code and utilise other Python modules such as Pandas / Numpy / Matplotlib or any other module you may be using.

Screenshot and provide an evaluation of its functionality so far.

Testing and Debugging

All throughout your project you should be explaining your testing and debugging with GitHub commits.

Once you have finished your program, have **at least two** other students perform a system-level test and give feedback based on your functional and non-functional requirements, response time, load testing and the suitability of the requirements.txt and README.md file.

Installation

Provide a clear README file describing how to run the app and any dependencies.

Include a correctly formatted requirements.txt file to make it easier for users to install.

Maintenance

Evaluate the role that maintenance would play in the continued implementation of this software.

Preliminary Software Engineering Assessment Task 1 Rubric - Part 1: Data Science Application (Practical)

/30	5 Marks (Per row)	4 Marks (Per row)	3 Marks (Per row)	2 Marks (Per row)	1-0 Marks (Per row)
API Data Retrieval	Successfully retrieves data from the API with error handling, ensuring all edge cases are addressed and logged.	Retrieves data from API, with most errors handled. Handles common edge cases.	Data retrieval successful with some errors; manages most key aspects but lacks depth in error handling.	Basic data retrieval; struggles with error handling and API responses.	Limited or no data retrieval; API errors not handled.
Error Handling & Validation	All user inputs are validated (e.g., spelling, case sensitivity); program prevents run-time errors and gives clear messages.	Most user inputs are validated, with meaningful error messages. Minor issues with run-time errors.	Input validation is adequate; some user errors may still lead to occasional run-time failures.	Basic input validation, frequent run-time errors, and unclear messages.	Limited or no error handling or validation implemented.
Use of Additional Python Modules	Outstanding use of multiple relevant modules, significantly enhancing functionality and efficiency.	Proficient use of relevant modules, improving functionality with minor areas for improvement.	Satisfactory use of basic modules, with some impact on functionality, but limited integration.	Basic use of modules, with minimal impact on functionality.	Limited or no use of additional modules, affecting program functionality.
Safe & Secure Programming	Implements secure coding practices, including data validation, exception handling, and safe API requests.	Mostly secure programming, with minor security concerns around validation or API requests.	Satisfactory security practices, but validation and exception handling could be improved.	Minimal security practices implemented; significant vulnerabilities present.	No security considerations; program is vulnerable to attacks or errors.
Code Documentation & ReadMe	Code is thoroughly commented, with a clear and informative README.md. Dependencies listed in requirements.txt.	Code is mostly commented; README.md includes essential information. Most dependencies listed.	Satisfactory comments and documentation but lacks some detail or clarity.	Minimal comments in code; README.md missing important instructions. Incomplete dependencies.	No comments, README.md, or dependency file present.
GitHub Version Control & Reflection	Regular and meaningful GitHub commits documenting changes and reflecting on decisions.	GitHub commits used, mostly documenting changes but lacking depth in reflection.	Satisfactory use of GitHub, though commits may lack regularity or detail.	Minimal GitHub use; commits are infrequent or insufficiently documented.	No GitHub version control or commits.
/20	10 Marks (Per row)	8 Marks (Per row)	6 Marks (Per row)	4 Marks (Per row)	2-0 Marks (Per row)
Code Structure	Code is highly efficient, with excellent use of functions, loops, and conditional statements. Redundant code is minimised and logically structured.	Code is well-organised with proficient use of functions, loops, and conditionals. Minor redundancy or inefficiencies present.	Code is satisfactorily structured with appropriate use of functions, loops, and conditionals. Some inefficiencies or repetition present.	Code uses basic functions, loops, and conditionals, but lacks efficiency or structure. Redundant sections are common.	Minimal or no use of functions, loops, and conditionals; code is inefficient and poorly organised.
User Interface Design	Graphical or text-based UI is implemented and is intuitive and responsive, with clear prompts and well-structured flow.	Graphical or text-based UI is implemented with clear prompts and well-structured flow.	Text-based UI satisfactory and functional, though lacks refinement or intuitive design.	Text-based UI is difficult to navigate, with poor prompt structure and unclear flow.	Limited functional user interface or prompts.

Preliminary Software Engineering Assessment Task 1 Rubric - Part 2: Data Science Application (Development Process)

/50	5 Marks (Per row)	4 Marks (Per row)	3 Marks (Per row)	2 Marks (Per row)	1-0 Marks (Per row)
Requirements Definition	Requirements are outstanding, fully verifiable, and cover all necessary features, properties, and behaviours.	Requirements are proficient, with most features, properties, and behaviours well-defined and verifiable.	Requirements are satisfactory, covering key elements but with some lacking detail or clarity.	Requirements are basic, with limited details, missing key features or verifiability.	Requirements are limited, unclear, and unverifiable.
Functional Specifications	Functional specifications are highly advanced, detailing clear input/output for each use case.	Functional specifications are advanced, with most input/output and use cases clearly defined.	Functional specifications are solid, covering the basics, but some details or use cases may be incomplete.	Functional specifications are elementary, missing several details or use cases.	Functional specifications are limited and incomplete.
Non-functional Specifications	Non-functional specifications are excellent, addressing both developer and user perspectives thoroughly.	Non-functional specifications are high, covering most key areas from both perspectives.	Non-functional specifications are adequate, but some details are missing or lacking depth.	Non-functional specifications are progressing, missing major aspects or perspectives.	Non-functional specifications are limited and vague.
Data Dictionary	Data dictionary is outstanding, with clearly defined and detailed variables, types, and descriptions.	Data dictionary is proficient, including most variables and providing adequate descriptions.	Data dictionary is satisfactory, but may be missing some variables or have unclear descriptions.	Data dictionary is basic, with limited variables or unclear descriptions.	No data dictionary or a limited one provided.
Structure Chart	Structure chart is highly advanced, accurately outlining all components and their relationships in the system.	Structure chart is advanced, with clear presentation of most components and relationships.	Structure chart is solid, but some components or relationships may be missing or unclear.	Structure chart is progressing, with many components or relationships missing or unclear.	Structure chart is limited or absent.
Flowcharts & Pseudocode	Flowcharts and pseudocode are outstanding, representing the program's logic with exceptional clarity and detail.	Flowcharts and pseudocode are proficient, representing the program's logic clearly, but may miss minor details.	Flowcharts and pseudocode are satisfactory, covering basic program logic, but lacking in clarity or detail.	Flowcharts and pseudocode are basic, with minimal or unclear representations of the program's logic.	Flowcharts and pseudocode are limited or not provided.
Gantt Chart for Scheduling	Gantt chart is outstanding, covering all tasks and milestones with accurate timelines and dependencies.	Gantt chart is proficient, covering most tasks and milestones with generally accurate timelines.	Gantt chart is satisfactory, but may lack some detail in task breakdown or timeline accuracy.	Gantt chart is basic, with limited tasks and unclear timelines.	No Gantt chart or a limited one provided.
Development: UI Prototyping & Progress Evaluation	UI is highly advanced, with all features functional. Progress is clearly documented, and an in-depth evaluation of current functionality and future development steps is included.	UI is advanced, with most features functional. Progress is documented with a clear evaluation of the current state and identified next steps for development.	UI is solid, with basic functionality demonstrated. Progress is somewhat documented, but evaluation and future steps may be lacking in detail.	UI is elementary, with minimal functionality demonstrated. Limited documentation of progress or future steps provided.	UI is limited, with no clear functionality or progress evaluation, and no clear plan for future steps.
Integration: API & Module Usage with Evaluation	API integration and module usage (e.g., Pandas) are highly advanced. Code functions smoothly, progress is well-documented, and a comprehensive evaluation of current functionality and future improvements is included.	API integration and module usage are advanced, with most components working well. Progress is documented, with a clear evaluation of current performance and planned future steps.	API integration and module usage are solid, with basic functionality in place. Progress is documented but lacks depth in the evaluation or next steps.	API integration is elementary, with limited functionality and minimal evaluation of progress or future steps.	API integration is limited, with little to no functionality and no evaluation or future planning.
Maintenance Evaluation	Maintenance needs are highly advanced, with a thorough evaluation of potential long-term issues, future updates, and improvements. Detailed plans for ongoing maintenance, bug fixing, and feature enhancements are included.	Maintenance evaluation is advanced, covering key areas such as future updates and bug fixing. There are clear plans for how the software will be maintained.	Maintenance evaluation is solid, with general considerations for future updates and bug fixes, though some details are missing or underdeveloped.	Maintenance evaluation is elementary, with minimal consideration for future updates or long-term maintenance.	Maintenance evaluation is limited, with little to no consideration for future maintenance or improvements.

